

Problem 2.

Part A. Assume f' is not an OWF and a p.p.t. algorithm $\mathcal{A}$ satisfies

$$\Pr[f'(x) = f'(\mathcal{A}(f'(x)))] \geq \frac{1}{|x|^c}.$$

Let $\mathcal{A}'(y) \leftarrow \mathcal{A}(y \| f(y))$. Then $f'(\mathcal{A}(f'(x))) = f'(x) \Rightarrow f(\mathcal{A}'(f(x))) = f(x)$. This contradicts the fact that $f(x)$ is an OWF. Therefore f' is an OWF.

Part B. Let $f_0 : \{0, 1\}^n \rightarrow \{0, 1\}^n$ be an OWF. Let $f : \{0, 1\}^{2n} \rightarrow \{0, 1\}^{2n}$, $|x_0| = |x_1|$, $f(x_0 \| x_1) = 0^n \| f_0(x_0)$. Then f is a length-preserving OWF. However, once $x \oplus f(x) = x_0 \| (x_1 \oplus f_0(x_0))$ is given, x can be recovered using $x_1 = (x_1 \oplus f_0(x_0)) \oplus f_0(x_0)$. So f' is not always an OWF.

Part D. Assume f' is not an OWF. Let $\mathcal{A}$ be a p.p.t. algorithm that satisfies

$$\Pr[f(G(x)) = f(G(\mathcal{A}(f(G(x)))))] \geq \frac{1}{|x|^c}.$$

Notice that when $X \sim U_{|x|+1}$, $\Pr[f(X) = f(G(\mathcal{A}(f(X))))] = \text{negl}(|x|)$ (since f is an OWF and $G \circ \mathcal{A}$ is a p.p.t. algorithm), while when $X \sim G(U_{|x|})$, the above probability $\geq \frac{1}{|x|^c}$. This gives contradiction. So f' is an OWF.

Part E. Let f_0 be an OWF and $f(x_0 \| x_1) = 0^n \| f_0(x_0)$. Let $f : \{0, 1\}^{2n} \rightarrow \{0, 1\}^{2n}$, $|x_0| = |x_1|$, $f(x_0 \| x_1) = 0^n \| f_0(x_0)$. Then f is a length-preserving OWF.

Let $G_0 : \{0, 1\}^{2n} \rightarrow \{0, 1\}^{2n+1}$ be a PRG. Let

$$G(x) = \begin{cases} 0^{2n+1}, & \text{when } x = 0^n \| x_1, \\ G_0(x), & \text{otherwise.} \end{cases}$$

Then G is still a PRG since the first case only occurs with negligible probability. In this case $G(f(x)) = 0^{2n+1}$ and is not an OWF.

Part G. Let $f_0 : \{0, 1\}^n \rightarrow \{0, 1\}^{(n-\log n)/2}$ be an OWF. Let

$$f(x) = \begin{cases} x, & \text{when } x = x_0 \| 0^{n-(n-\log n)/2}, \\ f_0(x) \| 0^{n-\log n-(n-\log n)/2} \| 1^{\log n}, & \text{otherwise.} \end{cases}$$

We can see that $f(x)_{1:(n-\log n)}$ is not an OWF (on y , an adversary can simply output y). On the other hand, assume f is not an OWF and we have a p.p.t. algorithm $\mathcal{A}$. Let $\mathcal{A}'(y) = \mathcal{A}(y \| 0^{n-\log n-(n-\log n)/2} \| 1^{\log n})$. Then except with negligible probability,

$$\mathcal{A}(f(x)) \in f^{-1}(f(x)) \Rightarrow \mathcal{A}'(f_0(x)) \in f_0^{-1}(f_0(x)).$$

So f is an OWF.

Problem 3.

Part A. When $\forall i, f(\hat{x}_i) = f(x_i)$, at least one of the following events happen:

- $\forall i, x_i$ is “good”. This happens w.p. $(\Pr_{x \leftarrow \{0,1\}^n}[x \text{ is “good”}])^m$.
- Some x_j is “bad”. In this case, we claim that

$$\Pr[\forall i, f(\hat{x}_i) = f(x_i)] \leq \frac{m}{r(n)}$$

where the probability is taken over $x_i (i \neq j)$ and $\mathcal{A}'$. If this claim is false, then during the execution of $\mathcal{A}(f(x))$, w.p. $\frac{1}{m}$ the i selected from $[m]$ equals j , so $\mathcal{A}$ success w.p. $\geq \frac{1}{m} \cdot \Pr[\forall i, f(\hat{x}_i) = f(x_i)] \geq \frac{1}{r(n)}$. This contradicts the assumption that x_j is “bad”. By taking union bound, the probability that some x_j is “bad” and $\forall i, f(\hat{x}_i) = f(x_i) \leq m \cdot \frac{m}{r(n)} = \frac{m^2}{r(n)}$. So $\Pr[\forall i, f(\hat{x}_i) = f(x_i)] \leq \frac{m^2}{r(n)} + (\Pr_{x \leftarrow \{0,1\}^n}[x \text{ is “good”}])^m$.

Part B. Assume $\Pr[x \text{ is “bad”}] > \frac{1}{2q(n)}$. From the conclusion in Part A, to prove contradiction it suffices to show that for sufficiently large n ,

$$\frac{1}{p(n)} \geq \frac{m^2}{r(n)} + (1 - \frac{1}{2q(n)})^m.$$

Pick $r(n) = 2m^2p(n)$, $m = 2nq(n)$. Then $\frac{m^2}{r(n)} = \frac{1}{2p(n)}$, and

$$(1 - \frac{1}{2q(n)})^m \leq \exp(-\frac{m}{2q(n)}) = e^{-n} < \frac{1}{2p(n)}$$

for sufficiently large n . So for infinitely many n ,

$$\frac{1}{p(n)} \geq \frac{m^2}{r(n)} + (1 - \frac{1}{2q(n)})^m.$$

This forces $\Pr[x \text{ is “bad”}] \leq \frac{1}{2q(n)}$ for infinitely many n .

Part C. When x is “good”, then the probability that $\mathcal{A}_{\text{repeat}}$ fails is at most

$$(1 - \frac{1}{r(n)})^{nr(n)} \leq e^{-n}.$$

So, conditioning on whether x is “good”,

$$\Pr_{x, \hat{x} \leftarrow \mathcal{A}_{\text{repeat}}(x)}[f(\hat{x}) \neq f(x)] \leq \frac{1}{2q(n)} + e^{-n} < \frac{1}{q(n)}$$

for infinitely many n . This violates the assumption on f .

Problem 4.

Part A. We claim that P' is not a PRP. Let $x_H, x'_H, x_L, x'_L \in \{0, 1\}^n$ be pairwise distinct. We have $w_L = ay_H + by_L$, where a, b are constants. So on we have

$$\begin{aligned} (x_H, x_L) &\mapsto (w_H, ay_H + by_L) \\ (x_H, x'_L) &\mapsto (w_H, ay_H + by'_L) \\ (x'_H, x_L) &\mapsto (w_H, ay'_H + by_L) \\ (x'_H, x'_L) &\mapsto (w_H, ay'_H + by'_L). \end{aligned}$$

Notice that the sum of w_L 's equals 0. However, this happens with only negligible probability in truly random permutations since the inputs are pairwise different. So P' is not a PRP.

Part B. It is obvious that P'' and the inverse function can be efficiently computed. We show that P'' is a PRP by providing a list of distinguish games. In each game, we let the adversary play against a different challenger. We show that consecutive games are indistinguishable for an adversary.

```

1  $k_1, k_2, k_3 \xleftarrow{R} \{0, 1\}^\lambda;$ 
2 On query  $(x_H, x_L)$ :
3    $y_H \leftarrow F(k_1, x_H);$ 
4    $y_L \leftarrow x_L;$ 
5    $(z_H, z_L)^\top \leftarrow M(y_H, y_L)^\top;$ 
6    $w_H \leftarrow F(k_2, z_H);$ 
7    $w_L \leftarrow F(k_3, z_L);$ 
8   return  $(w_H, w_L);$ 
```

Game 0

From the definition of PRP and a simple hybrid argument, we can replace $F(k_i, \cdot)$ with a random oracle $f_i(\cdot)$:

```

1  $f_1, f_2, f_3 \xleftarrow{R} \{0, 1\}^n \rightarrow \{0, 1\}^n;$ 
2 On query  $(x_H, x_L)$ :
3    $y_H \leftarrow f_1(x_H);$ 
4    $(z_H, z_L)^\top \leftarrow M(y_H, x_L)^\top;$ 
5    $w_H \leftarrow f_2(z_H);$ 
6    $w_L \leftarrow f_3(z_L);$ 
7   return  $(w_H, w_L);$ 
```

Game 1

A random function can be constructed by maintaining a dictionary L . On input x , if $L[x]$ is defined then return $L[x]$, otherwise set $L[x] \xleftarrow{R} \{0, 1\}^n$ and return $L[x]$.

```

1  $L_1, L_2, L_3 \leftarrow \emptyset;$ 
2  $L \leftarrow \emptyset;$ 
3 On query  $(x_H, x_L)$ :
4   if  $L[(x_H, x_L)]$  defined then
5      $\quad$  return  $L(x_H, x_L);$ 
6   if  $L_1[x_H]$  undefined then
7      $\quad$   $L_1[x_H] \xleftarrow{R} \{0, 1\}^n;$ 
8    $y_H \leftarrow L_1[x_H];$ 
9    $(z_H, z_L)^\top \leftarrow M(y_H, x_L)^\top;$ 
10  if  $L_2[z_H]$  undefined then
11     $\quad$   $L_2[z_H] \xleftarrow{R} \{0, 1\}^n;$ 
12   $w_H \leftarrow L_2[z_H];$ 
13  if  $L_3[z_L]$  undefined then
14     $\quad$   $L_3[z_L] \xleftarrow{R} \{0, 1\}^n;$ 
15   $w_L \leftarrow L_3[z_L];$ 
16   $L[(x_H, x_L)] \leftarrow (w_H, w_L);$ 
17  return  $(w_H, w_L);$ 

```

Game 2

Now we claim that except with negligible probability, $L_2[z_H]$ and $L_3[z_L]$ are never defined (in line 7 and line 10). When this event indeed happens, we trigger a *bad event*:

```

1  $L_1, L_2, L_3 \leftarrow \emptyset;$ 
2  $L \leftarrow \emptyset;$ 
3  $bad \leftarrow 0$ 
4 On query  $(x_H, x_L)$ :
5   if  $L[(x_H, x_L)]$  defined then
6      $\quad$  return  $L[(x_H, x_L)];$ 
7   if  $L_1[x_H]$  undefined then
8      $\quad$   $L_1[x_H] \xleftarrow{R} \{0, 1\}^n;$ 
9    $y_H \leftarrow L_1[x_H];$ 
10   $(z_H, z_L)^\top \leftarrow M(y_H, x_L)^\top;$ 
11  if  $L_2[z_H]$  defined then
12     $\quad$   $bad \leftarrow 1;$ 
13   $L_2[z_H] \xleftarrow{R} \{0, 1\}^n;$ 
14   $w_H \leftarrow L_2[z_H];$ 
15  if  $L_3[z_L]$  defined then
16     $\quad$   $bad \leftarrow 1;$ 
17   $L_3[z_L] \xleftarrow{R} \{0, 1\}^n;$ 
18   $w_L \leftarrow L_3[z_L];$ 
19   $L[(x_H, x_L)] \leftarrow (w_H, w_L);$ 
20  return  $(w_H, w_L);$ 

```

Game 3

If we fix all the randomness in Game 2 and Game 3 (it is required that in each query, the samples that $L_i[z]$ receive are the same in Game 2 and Game 3), then the outcome of the games are the same, as long as $bad = 0$ after the queries. Thus, the adversary can

distinguish the two challengers with advantage at most $\Pr[bad = 1]$ (it doesn't matter which game is the probability taken over).

Since w_H, w_L are just random numbers we can evaluate them in advance, and check bad after every query has been handled:

```

1   $L_1, L_2, L_3 \leftarrow \emptyset;$ 
2   $L \leftarrow \emptyset;$ 
3   $bad \leftarrow 0$ 
4  On query  $(x_H, x_L)$ :
5  |   if  $L[(x_H, x_L)]$  defined then
6  |   |   return  $L[(x_H, x_L)];$ 
7  |    $w_H \xleftarrow{R} \{0, 1\}^n;$ 
8  |    $w_L \xleftarrow{R} \{0, 1\}^n;$ 
9  |    $L[(x_H, x_L)] \leftarrow (w_H, w_L);$ 
10 |   return  $(w_H, w_L);$ 
11 for  $((x_H, x_L), (w_H, w_L)) \in L$  do
12 |   if  $L_1[x_H]$  undefined then
13 |   |    $L_1[x_H] \xleftarrow{R} \{0, 1\}^n;$ 
14 |    $y_H \leftarrow L_1[x_H];$ 
15 |    $(z_H, z_L)^\top \leftarrow M(y_H, x_L)^\top;$ 
16 |   if  $L_2[z_H]$  defined then
17 |   |    $bad \leftarrow 1;$ 
18 |    $L_2[z_H] \leftarrow w_H;$ 
19 |   if  $L_3[z_L]$  defined then
20 |   |    $bad \leftarrow 1;$ 
21 |    $L_3[z_L] \leftarrow w_L;$ 

```

Game 4

Now we show that $\Pr[bad = 1] = \text{negl}(n)$, so we only need to keep line 1~10 (which are exactly how a truly random permutation is generated).

Assume that there are $q(n)$ queries made by the adversary, where q is a fixed polynomial. Let $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$. The bad event happens in two cases.

- $z_H = z'_H \Leftrightarrow ay_H + bx_L = ay'_H + bx'_L$. Remember that $(x_H, x_L) \neq (x'_H, x'_L)$. If $x_H \neq x'_H$, then $\Pr[z_H = z'_H] = \frac{1}{2^n}$, since $a \neq 0$ and y_H, y'_H are independent. If $x_H = x'_H$, then $x_L \neq x'_L$ and $z_H \neq z'_H$. Taking union bound, this type of bad event happens w.p. $\leq \frac{q(n)^2}{2^n} = \text{negl}(n)$.
- $z_L = z'_L$. This is similar to the other case.

To summarize, we've shown that the bad event happens with negligible probability, so the adversary cannot distinguish with noticeable probability.

Problem 5.

Part A. We show that Feistel $_{f(k_1, \cdot), f(k_2, \cdot), f(k_3, \cdot), f(k_4, \cdot)}$ is a strong PRP by providing a list of distinguish games. In each game, we let the adversary play against a different challenger. We show that consecutive games are indistinguishable for an adversary.

```

1  $k_1, k_2, k_3 \xleftarrow{R} \{0, 1\}^\lambda;$ 
2 On Query( $x_0, x_1$ ):
3   for  $i \leftarrow 1$  to 4 do
4      $x_{i+1} \leftarrow x_{i-1} \oplus f(k_i, x_i);$ 
5   return ( $x_4, x_5$ );
6 On Query $^{-1}$ ( $x_4, x_5$ ):
7   for  $i \leftarrow 4$  downto 1 do
8      $x_{i-1} \leftarrow x_{i+1} \oplus f(k_i, x_i);$ 
9   return ( $x_0, x_1$ );

```

Game 0

Since f is a PRF, we can replace $f(k_i, \cdot)$ with truly random function $f_i(\cdot)$, which are implemented using dictionaries. We also need to save the results of *Query* and *Query* $^{-1}$. Let $\mathcal{X}$ consist of tuples $(x_0, x_1, x_4, x_5, b \in \{0, 1\})$.

```

1  $L_i \leftarrow \emptyset;$ 
2  $\mathcal{X} \leftarrow \emptyset;$ 
3 On Query( $x_0, x_1$ ):
4   if  $(x_0, x_1, \cdot, \cdot, \cdot) \in \mathcal{X}$  then
5      $(x_0, x_1, x_4, x_5, b) \leftarrow \mathcal{X};$ 
6     return ( $x_4, x_5$ );
7   for  $i \leftarrow 1$  to 4 do
8     if  $L_i[x_i]$  undefined then
9        $L_i[x_i] \xleftarrow{R} \{0, 1\}^n;$ 
10     $x_{i+1} \leftarrow x_{i-1} \oplus L_i[x_i];$ 
11   $\mathcal{X} \leftarrow \mathcal{X} \cup \{(x_0, x_1, x_4, x_5, 0)\};$ 
12  return ( $x_4, x_5$ );
13 On Query $^{-1}$ ( $x_4, x_5$ ):
14  if  $(\cdot, \cdot, x_4, x_5, \cdot) \in \mathcal{X}$  then
15     $(x_0, x_1, x_4, x_5, b) \leftarrow \mathcal{X};$ 
16    return ( $x_0, x_1$ );
17  for  $i \leftarrow 4$  downto 1 do
18    if  $L_i[x_i]$  undefined then
19       $L_i[x_i] \xleftarrow{R} \{0, 1\}^n;$ 
20     $x_{i-1} \leftarrow x_{i+1} \oplus L_i[x_i];$ 
21   $\mathcal{X} \leftarrow \mathcal{X} \cup \{(x_0, x_1, x_4, x_5, 1)\};$ 
22  return ( $x_0, x_1$ );

```

Game 1

We claim that w.h.p. in line 8 and 18, $L_2[x_2]$ and $L_3[x_3]$ are undefined, otherwise a bad event is triggered. When processing $Query$ ($Query^{-1}$), we also claim that $L_4[x_4]$ ($L_1[x_1]$) is undefined.

```

1  $L_i \leftarrow \emptyset$ ;
2  $\mathcal{X} \leftarrow \emptyset$ ;
3  $bad \leftarrow 0$ ;
4 On  $Query(x_0, x_1)$ :
5   if  $(x_0, x_1, \cdot, \cdot, \cdot) \in \mathcal{X}$  then
6      $(x_0, x_1, x_4, x_5, b) \leftarrow \mathcal{X}$ ;
7     return  $(x_4, x_5)$ ;
8   if  $L_1[x_1]$  undefined then
9      $L_1[x_1] \xleftarrow{R} \{0, 1\}^n$ ;
10   $x_2 \leftarrow x_0 \oplus L_1[x_1]$ ;
11  for  $i \leftarrow 2$  to 4 do
12    if  $L_i[x_i]$  defined then
13       $bad \leftarrow 1$ ;
14       $L_i[x_i] \xleftarrow{R} \{0, 1\}^n$ ;
15       $x_{i+1} \leftarrow x_{i-1} \oplus L_i[x_i]$ ;
16   $\mathcal{X} \leftarrow \mathcal{X} \cup \{(x_0, x_1, x_4, x_5, 0)\}$ ;
17  return  $(x_4, x_5)$ ;
18 On  $Query^{-1}(x_4, x_5)$ :
19   if  $(\cdot, \cdot, x_4, x_5, \cdot) \in \mathcal{X}$  then
20      $(x_0, x_1, x_4, x_5, b) \leftarrow \mathcal{X}$ ;
21     return  $(x_0, x_1)$ ;
22   if  $L_4[x_4]$  undefined then
23      $L_4[x_4] \xleftarrow{R} \{0, 1\}^n$ ;
24    $x_3 \leftarrow x_5 \oplus L_4[x_4]$ ;
25   for  $i \leftarrow 3$  downto 1 do
26     if  $L_i[x_i]$  defined then
27        $bad \leftarrow 1$ ;
28        $L_i[x_i] \xleftarrow{R} \{0, 1\}^n$ ;
29        $x_{i-1} \leftarrow x_{i+1} \oplus L_i[x_i]$ ;
30    $\mathcal{X} \leftarrow \mathcal{X} \cup \{(x_0, x_1, x_4, x_5, 1)\}$ ;
31   return  $(x_0, x_1)$ ;

```

Game 2

Now we can reorder the codes in line 11 $\sim$ 15 and 25 $\sim$ 29. That is, we sample x_i from $\{0,1\}^n$ and compute $L_i[x_i]$ from them. We can also move the output of a query earlier.

```

1   $L_i \leftarrow \emptyset$ ;
2   $\mathcal{X} \leftarrow \emptyset$ ;
3   $bad \leftarrow 0$ ;
4  On Query( $x_0, x_1$ ):
5      if  $(x_0, x_1, \cdot, \cdot, \cdot) \in \mathcal{X}$  then
6           $(x_0, x_1, x_4, x_5, b) \leftarrow \mathcal{X}$ ;
7          return  $(x_4, x_5)$ ;
8       $x_3, x_4, x_5 \xleftarrow{R} \{0, 1\}^n$ ;
9      if  $L_1[x_1]$  undefined then
10          $L_1[x_1] \xleftarrow{R} \{0, 1\}^n$ ;
11      $x_2 \leftarrow x_0 \oplus L_1[x_1]$ ;
12     for  $i \leftarrow 2$  to 4 do
13         if  $L_i[x_i]$  defined then
14              $bad \leftarrow 1$ ;
15          $L_i[x_i] \leftarrow x_{i-1} \oplus x_{i+1}$ ;
16      $\mathcal{X} \leftarrow \mathcal{X} \cup \{(x_0, x_1, x_4, x_5, 0)\}$ ;
17     return  $(x_4, x_5)$ ;
18 On Query-1( $x_4, x_5$ ):
19     if  $(\cdot, \cdot, x_4, x_5, \cdot) \in \mathcal{X}$  then
20          $(x_0, x_1, x_4, x_5, b) \leftarrow \mathcal{X}$ ;
21         return  $(x_0, x_1)$ ;
22      $x_0, x_1, x_2 \xleftarrow{R} \{0, 1\}^n$ ;
23     if  $L_4[x_4]$  undefined then
24          $L_4[x_4] \xleftarrow{R} \{0, 1\}^n$ ;
25      $x_3 \leftarrow x_5 \oplus L_4[x_4]$ ;
26     for  $i \leftarrow 3$  downto 1 do
27         if  $L_i[x_i]$  defined then
28              $bad \leftarrow 1$ ;
29          $L_i[x_i] \leftarrow x_{i-1} \oplus x_{i+1}$ ;
30      $\mathcal{X} \leftarrow \mathcal{X} \cup \{(x_0, x_1, x_4, x_5, 1)\}$ ;
31     return  $(x_0, x_1)$ ;

```

Game 3

Finally, we can return early and check every check for bad events after all the queries.

```

1  $L_i \leftarrow \emptyset$ ;
2  $\mathcal{X} \leftarrow \emptyset$ ;
3  $bad \leftarrow 0$ ;
4 On Query( $x_0, x_1$ ):
5   if  $(x_0, x_1, \cdot, \cdot, \cdot) \in \mathcal{X}$  then
6      $(x_0, x_1, x_4, x_5, b) \leftarrow \mathcal{X}$ ;
7     return  $(x_4, x_5)$ ;
8    $x_4, x_5 \xleftarrow{R} \{0, 1\}^n$ ;
9   if  $(\cdot, \cdot, x_4, \cdot, \cdot) \in \mathcal{X}$  then
10     $bad \leftarrow 1$ ;
11    $\mathcal{X} \leftarrow \mathcal{X} \cup \{(x_0, x_1, x_4, x_5, 0)\}$ ;
12   return  $(x_4, x_5)$ ;
13 On Query-1( $x_4, x_5$ ):
14   if  $(\cdot, \cdot, x_4, x_5, \cdot) \in \mathcal{X}$  then
15      $(x_0, x_1, x_4, x_5, b) \leftarrow \mathcal{X}$ ;
16     return  $(x_0, x_1)$ ;
17    $x_0, x_1 \xleftarrow{R} \{0, 1\}^n$ ;
18   if  $(\cdot, x_1, \cdot, \cdot, \cdot) \in \mathcal{X}$  then
19      $bad \leftarrow 1$ ;
20    $\mathcal{X} \leftarrow \mathcal{X} \cup \{(x_0, x_1, x_4, x_5, 1)\}$ ;
21   return  $(x_0, x_1)$ ;
22 for  $(x_0, x_1, x_4, x_5, b) \in \mathcal{X}$  do
23   if  $b = 0$  then
24     if  $L_1[x_1]$  undefined then
25        $L_1[x_1] \xleftarrow{R} \{0, 1\}^n$ ;
26      $x_2 \leftarrow x_0 \oplus L_1[x_1]$ ;
27      $x_3 \xleftarrow{R} \{0, 1\}^n$ ;
28     for  $i \leftarrow 2$  to 4 do
29       if  $L_i[x_i]$  defined then
30          $bad \leftarrow 1$ ;
31        $L_i[x_i] \leftarrow x_{i-1} \oplus x_{i+1}$ ;
32   else
33     if  $L_4[x_4]$  undefined then
34        $L_4[x_4] \xleftarrow{R} \{0, 1\}^n$ ;
35      $x_3 \leftarrow x_5 \oplus L_4[x_4]$ ;
36      $x_2 \xleftarrow{R} \{0, 1\}^n$ ;
37     for  $i \leftarrow 3$  downto 1 do
38       if  $L_i[x_i]$  defined then
39          $bad \leftarrow 1$ ;
40        $L_i[x_i] \leftarrow x_{i-1} \oplus x_{i+1}$ ;

```

Game 4

When $\mathcal{X}$ is fixed, we show that $\Pr[bad = 1] = \text{negl}(n)$. By symmetry, we only need to

analyze the $b = 0$ branch. Assume there are $q(n)$ total queries. The bad events on line 10 and 19 happen with negligible probability since x_4 and x_1 are sampled uniformly and there are only $q(n)^2/2$ pairs of queries.

- $x_2 = x'_2$. If $b' = 0$, $x_1 = x'_1$ (this is actually impossible since $x_0 \neq x'_0$) or $x_1 \neq x'_1$ and $L_1[x_1]$, $L_1[x'_1]$ are independent. So, the equality holds w.p. $\leq \frac{1}{2^n}$. If $b' = 1$, x'_2 is sampled uniformly, so the equality holds w.p. $\leq \frac{1}{2^n}$. By union bound the total collision probability is negligible.
- $x_3 = x'_3$. This happens with negligible probability since x_3 is sampled uniformly from $\{0, 1\}^n$.
- $x_4 = x'_4$. This won't happen since we've already checked.

Removing every sanity check and we have a truly strong random permutation. Since every adjacent challengers are indistinguishable, the 4-round Feistel network is a strong PRP.